

PRATICANDO

ENGENHARIA DE SOFTWARE



Apresentação

Praticar é fundamental para o seu aprendizado. Sentir-se desafiado, lidar com a frustração e aplicar conceitos são essenciais para fixar conhecimentos. No ambiente Praticando, você terá a oportunidade de enfrentar desafios específicos e estudos de caso, criados para ampliar suas competências e para a aplicação prática dos conhecimentos adquiridos.

Objetivo

Ampliar competências e consolidar conhecimentos através de desafios específicos e estudos de caso práticos.

Escolha do Modelo de Desenvolvimento de Software

Caso Prático

Carlos é o gerente de projeto de uma empresa de desenvolvimento de software em Belo Horizonte. Ele está liderando um projeto crítico que envolve a criação de um sistema de gestão de estoques para uma grande rede de supermercados. O projeto é complexo, com requisitos parcialmente identificados e alta volatilidade devido às frequentes mudanças solicitadas pelo cliente. A equipe de Carlos está pressionada a entregar funcionalidades operacionais rapidamente, mas o prazo é restrito e os recursos são limitados. A situação é agravada pelo fato de que a equipe é composta por profissionais com diferentes níveis de experiência, o que torna a comunicação e o alinhamento de expectativas desafiadores. Carlos precisa decidir qual modelo de processo de desenvolvimento de software adotar para equilibrar a necessidade de agilidade com a exigência de alta qualidade e controle no projeto.

Qual é o problema: A escolha do modelo de processo de desenvolvimento de software mais adequado para um projeto complexo e volátil.

Quem está vivenciando o problema: Carlos, o gerente de projeto, e sua equipe.

Onde o problema está ocorrendo: Em uma empresa de desenvolvimento de software em Belo Horizonte.

Quando o problema ocorre: Durante a fase de planejamento e definição de estratégias para o projeto de desenvolvimento do sistema de gestão de estoques.

Por que o problema ocorre: Devido à complexidade e volatilidade dos requisitos do projeto, à pressão por entregas rápidas e à necessidade de gerenciar uma equipe com diferentes níveis de experiência.

Diante da situação descrita, qual modelo de processo de desenvolvimento de software você recomendaria que Carlos adotasse para garantir o sucesso do projeto? Justifique sua escolha, considerando as características dos modelos de processo prescritivos e ágeis, e as necessidades específicas do projeto.

Chave de resposta

Para o projeto de Carlos, o modelo de desenvolvimento ágil, como o Scrum ou o Extreme Programming (XP), seria a escolha mais adequada. Esses modelos são projetados para lidar com a alta volatilidade dos requisitos, permitindo entregas incrementais e ajustáveis, o que é essencial dado o contexto de constantes mudanças solicitadas pelo cliente. O Scrum, por exemplo, facilita a organização do trabalho em sprints curtos, promovendo revisões contínuas e entregas rápidas, o que atenderia à necessidade de funcionalidades operacionais entregues em prazos restritos.

Além disso, a inclusão de práticas ágeis, como a comunicação frequente e a colaboração entre os membros da equipe, pode ajudar a mitigar os desafios relacionados à variabilidade de experiência entre os profissionais. O XP, com sua ênfase na programação em par e no feedback constante, poderia ajudar a elevar o nível técnico da equipe menos experiente, garantindo um código de alta qualidade.

No entanto, deve-se considerar a integração de práticas de controle e qualidade típicas de modelos prescritivos, como o RUP, especialmente em fases críticas onde a documentação e a estabilidade são cruciais. Essa combinação híbrida permitiria que Carlos mantivesse a agilidade necessária para responder às mudanças, ao mesmo tempo em que asseguraria a qualidade e a conformidade com os padrões esperados pelo cliente.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

“O engenheiro de software deverá definir qual o processo de desenvolvimento a ser aplicado em um determinado projeto de software como especificação de requisito não funcional. Inicialmente, terá que identificar quais as atividades que irão compor o desejado processo e, em seguida, definir o sequenciamento das referidas atividades, ou seja, o fluxo do processo.

As atividades típicas que compõem o processo de desenvolvimento de software estão ilustradas na figura 4. O objetivo é ilustrar as atividades mais comuns que compõem os processos de desenvolvimento de software, ou seja, qualquer processo deverá possuir as referidas atividades”.

Fundamentos de Software e Gerenciamento de Projetos

Desafio 1

Como um engenheiro de software em um ambiente corporativo, você precisa entender e aplicar diferentes modelos de ciclo de vida de desenvolvimento de software. Esses modelos não apenas guiam a construção e entrega de sistemas, mas também influenciam diretamente a gestão de riscos, a satisfação do cliente e a eficiência do projeto. Considere o seguinte cenário: você foi incumbido de escolher o modelo mais adequado para um novo projeto de software, que envolve altos riscos técnicos e a necessidade de entregar incrementos ao cliente de forma contínua. Com base em seu conhecimento sobre os modelos de ciclo de vida, analise as afirmativas a seguir:

I. O desenvolvimento em cascata sugere a criação de uma implementação inicial, revisada e aprimorada em versões subsequentes com base no feedback do usuário, até que o sistema esteja completo.

II. O modelo espiral enfatiza a avaliação contínua dos riscos em cada fase do ciclo de desenvolvimento, buscando mitigar problemas antes que eles impactem negativamente o projeto.

III. O Rapid Application Development (RAD) acelera o processo de desenvolvimento utilizando componentes pré-fabricados, uma adaptação do modelo cascata com ciclos de desenvolvimento mais rápidos.

IV. O modelo incremental permite que os clientes especifiquem as funções do sistema e as recebam em etapas, onde cada entrega contém uma parte das funcionalidades do sistema final.

Com relação às afirmativas, estão corretas:

A

Somente as afirmativas I e II são corretas.

B

Somente as afirmativas I e III são corretas.

C

Somente as afirmativas III e IV são corretas.

D

Somente as afirmativas I, II e IV são corretas.

E

Somente as afirmativas II, III e IV são corretas.



A alternativa E está correta.

O modelo espiral (afirmativa II) é conhecido por sua ênfase na gestão de riscos, onde cada iteração inclui uma análise detalhada dos riscos e a criação de protótipos, que ajudam a evitar falhas graves no projeto. O RAD (afirmativa III) é de fato uma variação do modelo cascata, com foco em ciclos rápidos de desenvolvimento usando componentes pré-existentes, o que facilita entregas mais rápidas. O modelo incremental (afirmativa IV) permite entregas em fases, cada uma contendo parte das funcionalidades finais, atendendo gradualmente as necessidades do cliente enquanto o projeto evolui. A afirmativa I está incorreta porque descreve erroneamente o modelo cascata; o que foi descrito é mais próximo de um modelo iterativo e não do tradicional cascata, onde todas as fases são completadas sequencialmente sem revisões.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Processo de Desenvolvimento de Software

“O processo de desenvolvimento de software inclui algumas etapas básicas, cujo momento de aplicação depende do modelo adotado pelo engenheiro de software... Um requisito importante na seleção de um processo de software é a complexidade. Basicamente, quanto maior a complexidade de um sistema, mais formal deve ser o processo adotado. Importante: a qualidade é a camada base que sustenta a camada processo.”

Desafio 2

Imagine que você, como gerente de projetos de uma empresa de tecnologia, está liderando o desenvolvimento de um novo software crítico para um cliente importante. Você sabe que aplicar uma metodologia sistemática é importante para garantir o sucesso do projeto. Nesse contexto, surge a necessidade de entender como o Project Management Institute (PMI) pode ser um recurso valioso.

Diante disso, reflita sobre o papel do PMI e assinale qual das seguintes alternativas descreve corretamente sua função.

A

O Project Management Institute (PMI) é uma certificadora de PMP.

B

O Project Management Institute (PMI) é uma ferramenta de gestão de projetos.

C

O Project Management Institute (PMI) é uma forma de PMBOK.

D

O Project Management Institute (PMI) é uma estrutura de apoio gerencial.

E

O Project Management Institute (PMI) é um instrumento de elaboração de hardware.



A alternativa A está correta.

A) O Project Management Institute (PMI) é uma certificadora de PMP: Correta. O Project Management Institute (PMI) é uma organização internacionalmente reconhecida, que promove o profissionalismo no gerenciamento de projetos. Um dos principais serviços oferecem é a certificação PMP (Project Management Professional), que é uma das certificações mais respeitadas na área de gerenciamento de projetos. Essa certificação é destinada a profissionais que desejam validar suas habilidades e conhecimentos em gerenciar projetos de forma eficaz e eficiente, segundo os padrões globais estabelecidos pelo PMI.

B) O Project Management Institute (PMI) é uma ferramenta de gestão de projetos: Incorreta. Embora o PMI desenvolva ferramentas e técnicas que são amplamente usadas na gestão de projetos, ele próprio não é uma ferramenta. O PMI é uma instituição que estabelece padrões, guia melhores práticas e oferece certificações para profissionais. Ferramentas de gestão de projetos, como softwares específicos, são criadas com base nas diretrizes e padrões desenvolvidos por instituições como o PMI, mas a organização em si não deve ser confundida com essas ferramentas.

C) O Project Management Institute (PMI) é uma forma de PMBOK: Incorreta. O PMBOK (Project Management Body of Knowledge) é um guia publicado pelo PMI, que compila as melhores práticas e padrões reconhecidos no gerenciamento de projetos. Portanto, o PMBOK é um produto do PMI, e não uma forma dele. Essa alternativa confunde a entidade organizacional com um de seus principais recursos publicados.

D) O Project Management Institute (PMI) é uma estrutura de apoio gerencial: Incorreta. Embora o PMI forneça uma estrutura de apoio através de padrões, certificações e diretrizes, a descrição como uma “estrutura de apoio gerencial” é vaga e não captura adequadamente a função principal do PMI. A função do PMI vai além de simplesmente fornecer apoio; ele define as normas globais e certifica profissionais em práticas de gestão de projetos.

E) O Project Management Institute (PMI) é um instrumento de elaboração de hardware: Incorreta. Esta alternativa está totalmente errada, pois confunde o propósito do PMI. O PMI não se envolve na criação ou desenvolvimento de hardware; sua missão é focada no desenvolvimento de padrões e certificações para o gerenciamento de projetos, independentemente do tipo de projeto ou da indústria.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Gerenciamento de Projeto

“Agora que você tem uma boa compreensão das atividades genéricas que compõem um processo e as possibilidades de fluxos de processos... O PMI, Project Management Institute, é uma organização que visa disseminar as melhores práticas de Gerenciamento de Projetos em todo o mundo e que tem como premissa o fato de ser esse gerenciamento uma profissão.”

Desafio 3

Como profissional de desenvolvimento de software, você será frequentemente responsável por gerenciar o ciclo de vida de projetos complexos, onde o entendimento profundo dos fluxos de processo é essencial. Imagine-se liderando um projeto crítico em uma empresa de tecnologia, onde o sucesso depende do alinhamento preciso entre as atividades sequenciais e paralelas. Para garantir a eficiência e a entrega de um produto de alta qualidade, é importante compreender como as atividades do ciclo de vida do processo se relacionam e se integram. Considerando essa responsabilidade, analise as seguintes afirmações sobre os fluxos de processo no desenvolvimento de software e escolha a alternativa correta:

I - No fluxo de processo linear, as atividades são executadas em sequência de forma unidirecional.

Porque

II - No fluxo de processo paralelo, as atividades podem ocorrer em paralelo.

Marque a alternativa que apresenta a relação correta entre as asserções I e II:

A

As duas afirmativas estão corretas e a segunda não justifica a primeira.

B

As duas afirmativas estão corretas e a segunda justifica a primeira.

C

As duas afirmativas são falsas.

D

A primeira afirmativa é verdadeira e a segunda afirmativa é falsa.

E

A primeira afirmativa é falsa e a segunda afirmativa é verdadeira.



A alternativa A está correta.

No gerenciamento de processos de software, é importante entender que diferentes tipos de fluxo podem ser aplicados dependendo do contexto do projeto. O fluxo linear, também conhecido como "cascata", envolve a execução das atividades de forma sequencial, onde uma etapa só começa quando a anterior é

concluída. Este modelo é apropriado para projetos onde os requisitos são bem definidos desde o início e mudanças são raras. Por outro lado, o fluxo paralelo permite que várias atividades ocorram simultaneamente, o que é útil em projetos que necessitam de alta eficiência e rapidez. No entanto, a capacidade de realizar atividades em paralelo não justifica a natureza unidirecional do fluxo linear; são abordagens distintas que podem ser aplicadas de acordo com as necessidades do projeto.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Processo de Desenvolvimento de Software

“O processo de desenvolvimento de software inclui diversas etapas, e a especificação de um processo de desenvolvimento de software requer a definição de quais atividades irão compor o respectivo processo e como essas atividades serão encadeadas. Este encadeamento é conhecido como Fluxo de Processo ou Ciclo de Vida. Entre os tipos de fluxos, destacam-se o fluxo linear, onde as atividades são realizadas em sequência, e o fluxo paralelo, onde as atividades podem ocorrer simultaneamente, possibilitando um tratamento mais dinâmico e eficiente em certos projetos.”

Desafio 4

Como gestor de uma empresa de desenvolvimento de software, você precisa tomar decisões estratégicas sobre quais projetos serão priorizados no portfólio de sua empresa. Considerando que a escolha do portfólio afeta diretamente o sucesso da organização no mercado competitivo, é essencial avaliar os fatores cruciais para essa decisão. Qual dos fatores listados a seguir é mais determinante na seleção de um portfólio de projetos de software?

A

Velocidade de entrega.

B

Estimativa de lucro.

C

Viabilidade técnica.

D

Análise de mercado.

E

Tamanho da equipe.



A alternativa C está correta.

A) Velocidade de entrega: Incorreta. A velocidade de entrega é um fator importante em projetos de software, especialmente em mercados competitivos onde o time-to-market pode determinar o sucesso ou fracasso de um produto. No entanto, priorizar a velocidade de entrega sem considerar a viabilidade técnica pode levar a resultados de baixa qualidade ou mesmo ao fracasso do projeto. A entrega rápida de um software tecnicamente inviável compromete a sustentabilidade e a eficácia do portfólio de projetos.

B) Estimativa de lucro: Incorreta. A estimativa de lucro é, sem dúvida, um aspecto vital na seleção de projetos. No entanto, projetos com alta estimativa de lucro que não são tecnicamente viáveis dificilmente serão sustentáveis no longo prazo. A viabilidade técnica assegura que o projeto pode ser executado dentro dos parâmetros técnicos disponíveis, garantindo que os lucros previstos possam realmente ser alcançados.

C) Viabilidade técnica: Correta. A viabilidade técnica é o fator mais importante na seleção de um portfólio de projetos de software. Antes de qualquer consideração sobre lucro ou tempo de entrega, é essencial garantir que o projeto pode ser tecnicamente implementado com os recursos disponíveis. A viabilidade técnica envolve a avaliação da arquitetura de software, a tecnologia utilizada, e a capacidade da equipe de desenvolvimento para entregar o projeto com sucesso. Se um projeto não for tecnicamente viável, ele não só falhará em entregar valor, mas também poderá causar prejuízos significativos à empresa.

D) Análise de mercado: Incorreta. A análise de mercado é um componente importante ao considerar quais projetos devem ser incluídos em um portfólio, pois determina a demanda e a aceitação do produto no mercado. Contudo, sem a viabilidade técnica, um projeto bem posicionado no mercado ainda pode fracassar se não puder ser entregue com a qualidade e funcionalidade necessárias. A viabilidade técnica garante que a análise de mercado possa ser efetivamente traduzida em um produto de sucesso.

E) Tamanho da equipe: Incorreta. Embora o tamanho da equipe seja um fator importante para planejar a alocação de recursos e definir prazos, ele não deve ser o principal critério na seleção de projetos. Uma equipe maior ou menor pode ser ajustada dependendo da complexidade técnica e dos recursos necessários, mas a viabilidade técnica deve sempre ser garantida primeiro para que o projeto possa ser realizado com sucesso.

Para saber mais sobre esse conteúdo, acesse:

Módulo 1

Introdução

“Uma área de conhecimento importante, aplicada em qualquer engenharia, é o Gerenciamento de Projeto... A Engenharia de Software está alinhada com a área de conhecimento Gerenciamento de Projetos, sendo esta fundamental ao engenheiro de software.”

Fases do Desenvolvimento de Software

Desafio 1

Como profissional na área de desenvolvimento de software, você se encontrará frequentemente na etapa de projeto, onde precisará refinar modelos de análise e estruturar a arquitetura de sistemas para garantir que todas as funcionalidades sejam implementadas de forma eficiente e escalável. Imagine que você está liderando o projeto de um novo sistema, e é necessário tomar decisões críticas que irão afetar a arquitetura e

a qualidade final do produto. Essas decisões envolvem a compreensão profunda dos conceitos de engenharia de software, desde a abstração dos modelos até a definição das camadas que compõem a estrutura lógica e física do sistema.

Considere as seguintes afirmativas relacionadas com a etapa de projeto do processo de desenvolvimento de software:

I. A partir do modelo de classes gerado na análise, aplicam-se refinamentos que permitem reduzir o grau de abstração do referido modelo.

II. Os aspectos dinâmicos, considerando que o paradigma dominante na indústria de software é a orientação a objetos, são definidos nos modelos de classes.

III. A divisão lógica comumente aplicada no projeto de arquitetura pode ser composta das seguintes camadas: apresentação, aplicação, domínio e serviços técnicos.

Assinale a alternativa correta:

A

Apenas as afirmativas I e III são verdadeiras.

B

Apenas a afirmativa II é verdadeira.

C

Apenas a afirmativa I é verdadeira.

D

Apenas as afirmativas II e III são verdadeiras.

E

As afirmativas I, II e III são verdadeiras.



A alternativa A está correta.

No contexto do desenvolvimento de software, especialmente na fase de projeto, é importante entender como os modelos de classes evoluem da análise para a implementação. A afirmativa I está correta porque, na fase de projeto, o engenheiro de software deve refinar os modelos de classes gerados na análise, inserindo detalhes que diminuem a abstração e possibilitam a codificação. Esses refinamentos envolvem a adição de atributos, métodos, e associações entre classes, garantindo que a especificação seja precisa e implementável.

A afirmativa III também está correta ao mencionar a divisão lógica da arquitetura em camadas. Essa abordagem permite a decomposição do sistema em partes menores, facilitando a manutenção e o

desenvolvimento futuro. As camadas de apresentação, aplicação, domínio e serviços técnicos são essenciais para criar uma arquitetura robusta e modular, onde cada camada tem responsabilidades claramente definidas e pode ser desenvolvida de forma independente.

Por outro lado, a afirmativa II é incorreta. Os aspectos dinâmicos, que envolvem a interação entre objetos e a comunicação através de mensagens, são normalmente modelados em diagramas de interação, como os diagramas de sequência ou de comunicação, e não nos modelos de classes. Assim, a afirmativa II não reflete com precisão a natureza do trabalho realizado na fase de projeto.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Projeto de Software

"Refinamento dos Aspectos Estáticos e Estruturais do Sistema: O modelo de classes representa os aspectos estáticos e estruturais do sistema. A partir do modelo de classes gerado na análise, vamos inserir novos elementos que permitirão a implementação das classes, ou seja, aplicaremos refinamentos que possibilitam reduzir o grau de abstração. A arquitetura é definida por meio de duas abstrações: lógica e física. A divisão lógica comumente aplicada no projeto de arquitetura é composta das seguintes camadas: apresentação, aplicação, domínio e serviços técnicos."

Desafio 2

Você é um engenheiro de software responsável por desenvolver a interface gráfica de um novo sistema. Sabe que a qualidade da interface é importante para a aceitação do sistema pelos usuários finais. Entre os requisitos do projeto, você precisa identificar qual dos requisitos não funcionais deve ser priorizado ao definir a interface gráfica com o usuário, considerando aspectos como usabilidade e acessibilidade.

Qual é o requisito-chave não funcional ao definir a interface gráfica com o usuário?

A

Funcionalidade.

B

Desempenho.

C

Segurança.

D

Integridade.

E

Usabilidade.



A alternativa E está correta.

A) Funcionalidade: Incorreta. A funcionalidade é essencial para garantir que o sistema cumpra suas funções previstas, mas não é o foco principal ao definir uma interface gráfica. A funcionalidade refere-se ao conjunto de operações que o software pode realizar, mas a interface gráfica precisa, sobretudo, ser intuitiva e fácil de usar, o que é abordado pelo conceito de usabilidade.

B) Desempenho: Incorreta. O desempenho se refere à capacidade do sistema de operar sob determinadas condições de carga e resposta. Embora importante, o desempenho não está diretamente relacionado ao design da interface gráfica, mas sim à eficiência do software em executar suas funções, como tempo de resposta e processamento.

C) Segurança: Incorreta. A segurança é um requisito não funcional que protege o sistema contra acessos não autorizados e falhas. Embora importante em muitos contextos, a segurança não é o requisito principal ao desenvolver uma interface gráfica, pois o foco na interface está em como os usuários interagem com o sistema.

D) Integridade: Incorreta. Integridade refere-se à consistência e precisão dos dados dentro do sistema. Embora seja um aspecto importante da qualidade do software, ele não está diretamente relacionado ao design ou à experiência do usuário na interface gráfica.

E) Usabilidade: Correta. Usabilidade é o requisito-chave ao definir a interface gráfica de um sistema. Ela assegura que a interface seja intuitiva, fácil de aprender e eficiente para o usuário. A usabilidade está diretamente relacionada à satisfação do usuário, facilitando a navegação e a realização de tarefas sem erros ou frustrações. Neste contexto, uma interface gráfica bem desenhada melhora a experiência do usuário, promovendo maior adesão ao sistema e reduzindo a necessidade de suporte técnico.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Projeto de Software

"O diagrama de casos de uso define as funcionalidades do sistema, sendo que cada caso de uso representa uma interação completa entre o sistema e um agente externo denominado ator. Nessa interação, precisamos definir uma interface como meio de comunicação homem-máquina. Nesse ponto, o requisito-chave não funcional é usabilidade, portanto é fundamental a participação do usuário no processo de definição das interfaces, ou seja, 'deixe o usuário no comando'".

Desafio 3

Você está liderando o desenvolvimento de um aplicativo educacional e, como parte do processo de projeto, deve definir uma arquitetura de sistema que suporte a manutenção e a escalabilidade futuras. A arquitetura deve permitir que o sistema se adapte a novos requisitos sem comprometer a estabilidade. Como profissional,

você entende que a escolha das atividades de projeto é importante para garantir que o software atenda a essas necessidades.

Qual atividade é essencial na fase de projeto deste aplicativo educacional?

A

Escolha do sistema de gerenciamento de banco de dados.

B

Definição do modelo de classes do sistema.

C

Codificação das funcionalidades principais.

D

Testes de usabilidade com usuários finais.

E

Implantação de módulos educacionais.



A alternativa B está correta.

A) Escolha do sistema de gerenciamento de banco de dados: Incorreta. A escolha do sistema de gerenciamento de banco de dados é uma decisão técnica importante, mas que ocorre tipicamente após a definição da arquitetura e do modelo de classes. Ela não é uma atividade central na fase inicial do projeto, onde o foco está em estruturar o sistema de forma que ele possa ser mantido e escalado.

B) Definição do modelo de classes do sistema: Correta. A definição do modelo de classes é importante na fase de projeto porque fornece uma estrutura clara e detalhada dos elementos principais do sistema. O modelo de classes mapeia os objetos de domínio, suas relações e comportamentos, servindo como base para a codificação e testes subsequentes. Esse modelo é importante para garantir que o sistema seja flexível e adaptável a novas funcionalidades, além de suportar a manutenção e a escalabilidade com eficiência.

C) Codificação das funcionalidades principais: Incorreta. A codificação é uma etapa que ocorre após a fase de projeto, quando os modelos e especificações já foram estabelecidos. Codificar sem uma base sólida na arquitetura e no modelo de classes pode resultar em um software mal estruturado, difícil de manter e escalar.

D) Testes de usabilidade com usuários finais: Incorreta. Embora importante, os testes de usabilidade ocorrem geralmente após a implementação, quando a interface e as funcionalidades principais já foram desenvolvidas. Eles não são uma atividade essencial na fase inicial de projeto.

E) Implantação de módulos educacionais: Incorreta. A implantação é a fase final do desenvolvimento, onde o software é distribuído para o ambiente de produção. Antes disso, é essencial garantir que o projeto e o modelo de classes estejam bem definidos para que a implantação seja bem-sucedida.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Projeto de Software

"O modelo de classes representa os aspectos estáticos e estruturais do sistema. A partir do modelo de classes gerado na análise, vamos inserir novos elementos que permitirão a implementação das classes, ou seja, aplicaremos refinamentos que possibilitam reduzir o grau de abstração.".

Desafio 4

Imagine que você está trabalhando na implementação de um novo software de gestão empresarial. Sua equipe é responsável por transformar os modelos detalhados de projeto em código executável, garantindo que os padrões de qualidade sejam mantidos durante o desenvolvimento. Você sabe que a tradução precisa dos modelos em código é essencial para o sucesso do projeto e deve ser priorizada para evitar problemas futuros.

Qual é o principal foco da equipe durante a fase de implementação deste software de gestão empresarial?

A

Análise de requisitos do sistema.

B

Design de interface do usuário.

C

Tradução dos modelos de projeto em código.

D

Planejamento da infraestrutura de rede.

E

Teste de integração de sistemas.



A alternativa C está correta.

A) Análise de requisitos do sistema: Incorreta. A análise de requisitos é uma atividade que precede a fase de implementação. Ela ocorre na fase inicial do desenvolvimento de software e tem como objetivo

identificar e documentar o que o sistema deve fazer. Na fase de implementação, o foco é transformar os modelos de projeto baseados nesses requisitos em código funcional.

B) Design de interface do usuário: Incorreta. O design da interface do usuário é uma parte importante do processo de desenvolvimento, mas não é o principal foco durante a fase de implementação. O design da interface deve ser finalizado antes da codificação para que os desenvolvedores possam seguir o plano ao escrever o código.

C) Tradução dos modelos de projeto em código: Correta. A tradução dos modelos de projeto em código é o principal foco durante a fase de implementação. Nesta fase, os desenvolvedores convertem as especificações detalhadas do projeto em código executável, garantindo que o sistema funcione conforme planejado. Esse processo é crítico, pois qualquer desvio pode resultar em um software que não atende aos requisitos de negócios ou que apresenta falhas de desempenho.

D) Planejamento da infraestrutura de rede: Incorreta. O planejamento da infraestrutura de rede é uma consideração importante, mas normalmente é tratado separadamente da fase de implementação do software. Ele envolve questões de hardware e conectividade, que, embora essenciais para o funcionamento do software, não são o foco principal durante a codificação.

E) Teste de integração de sistemas: Incorreta. O teste de integração é uma atividade que ocorre após a implementação inicial do código. Ele é necessário para garantir que diferentes componentes do sistema funcionem juntos como esperado, mas não é o foco durante a codificação inicial.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Implementação

"A etapa 'implementação' do processo de desenvolvimento de software permite realizar a tradução dos modelos de projeto em código executável por meio do uso de uma ou mais linguagens de programação."

Modelos de processos de desenvolvimento de software

Desafio 1

Imagine que você está atuando como um desenvolvedor de software em uma equipe que adota práticas de Extreme Programming (XP). Durante uma reunião de planejamento de iteração, você precisa participar ativamente na definição e estimativa das tarefas que serão realizadas na próxima iteração. Esse processo, conhecido como Iteration Planning Game, é importante para o sucesso do projeto, pois envolve a escolha das atividades que serão implementadas com base na complexidade e na prioridade dos requisitos. Dentre as atividades realizadas nessa sessão, qual delas é normalmente abordada?

A

Definição, pelos programadores, de quais story cards serão implementados em uma iteração.

B

Estimação do esforço que será necessário para implementar cada story card.

C

Estimação da data de entrega de um release baseado na estimativa de esforço de cada story card.

D

Estimação, feita por cada programador, do tempo que será necessário para realizar cada tarefa sob sua responsabilidade.

E

Designação, por parte do coach, dos programadores que irão realizar as tarefas contidas na lista de tarefas.



A alternativa D está correta.

A) Definição, pelos programadores, de quais story cards serão implementados em uma iteração: Incorreta. Embora a definição dos story cards seja uma parte importante do planejamento, a tarefa de estimar o tempo necessário para realizar cada tarefa é feita individualmente pelos programadores após a escolha dos cards. A definição dos story cards depende das prioridades estabelecidas em conjunto com o cliente e da avaliação do esforço necessário.

B) Estimação do esforço que será necessário para implementar cada story card: Incorreta. A estimativa do esforço é realizada, mas o que a questão busca é a atividade específica em que cada programador estima o tempo necessário para completar suas tarefas individuais, o que não está exatamente contemplado aqui.

C) Estimação da data de entrega de um release baseado na estimativa de esforço de cada story card: Incorreta. Estimar a data de entrega de um release é uma atividade posterior e mais abrangente, que depende da soma dos esforços estimados para cada story card, mas não é o foco da atividade de estimativa individual de tempo.

D) Estimação, feita por cada programador, do tempo que será necessário para realizar cada tarefa sob sua responsabilidade: Correta. Esta é a resposta correta, pois no Iteration Planning Game, cada programador estima o tempo necessário para realizar as tarefas que lhes foram atribuídas. Isso é essencial para que o time tenha uma noção precisa do que pode ser completado na iteração.

E) Designação, por parte do coach, dos programadores que irão realizar as tarefas contidas na lista de tarefas: Incorreta. O coach pode facilitar o processo e ajudar a equipe a tomar decisões, mas a designação específica de tarefas é uma responsabilidade compartilhada entre os membros do time, e não imposta de forma unilateral.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Explicar o processo de desenvolvimento ágil Extreme Programming (XP)

“O planejamento está fundamentado nas histórias de usuários. O projeto tem um mínimo de formalismo em função de gerar um único tipo de artefato, os cartões CRC, sendo sugeridas a prototipação e refatoração nessa etapa. A codificação sugere o trabalho em par de programadores alocados em um único computador, permitindo uma revisão de código em tempo real. Os testes têm uma característica peculiar, são escritos antes da codificação e aplicados de forma sistemática em ambiente automatizado; na sequência, temos os testes unitários, de integração e aceitação.”

Desafio 2

Imagine que você é um desenvolvedor de software responsável por um projeto onde a prototipação é uma técnica importante. Durante uma reunião, sua equipe discute a necessidade de aplicar um processo que envolva quatro etapas essenciais para a criação de protótipos, assegurando que o produto final atenda às expectativas do cliente. Como profissional experiente, você precisa decidir a ordem correta dessas etapas para garantir que o protótipo seja desenvolvido de forma eficaz e com o menor risco possível de retrabalho. Considerando os princípios da Engenharia de Software e a importância de um processo bem estruturado, analise a sequência adequada para a execução dessas etapas no desenvolvimento de protótipos.

- I. Avaliar protótipo.
- II. Estabelecer objetivos do protótipo.
- III. Desenvolver o protótipo.
- IV. Definir a funcionalidade do protótipo.

A ordem sequencial correta para a execução dessas quatro etapas é:

A

I, IV, II e III.

B

II, IV, III e I.

C

III, II, IV e I.

D

IV, I, II e III.

E

IV, II, I e III.



A alternativa B está correta.

A aplicação correta das etapas da prototipação é essencial para o sucesso de um projeto de software, especialmente em ambientes onde a volatilidade dos requisitos é alta. A sequência correta começa com o estabelecimento dos objetivos do protótipo (II), o que permite à equipe compreender o propósito do protótipo e definir claramente o que se espera alcançar. Em seguida, é necessário definir a funcionalidade do protótipo (IV), garantindo que todos os requisitos e funcionalidades essenciais sejam especificados antes do desenvolvimento. Após essa definição, a equipe pode desenvolver o protótipo (III), implementando as funcionalidades identificadas. Finalmente, avaliar o protótipo (I) permite verificar se ele atende aos objetivos inicialmente estabelecidos e identificar possíveis melhorias. Esse ciclo iterativo ajuda a refinar continuamente o protótipo até que ele atenda às expectativas do cliente e às necessidades do projeto.

Para saber mais sobre esse conteúdo, acesse:

Módulo 1

Modelos de Processos Prescritivos

"Na prototipação, o desenvolvedor interage diretamente com o usuário, escutando seus pedidos e desenvolvendo, imediatamente, um protótipo do produto desejado, isso de forma iterativa e evolucionária... Podemos descartar esse software gerado? Depende, normalmente essa primeira versão do protótipo serve apenas para validar um requisito e necessita ser reconstruída, mas, considerando as peculiaridades do software, essa mesma versão pode ser refinada e integrada ao sistema em desenvolvimento."

Este trecho ressalta a importância das etapas sequenciais na prototipação, enfatizando que o desenvolvimento e a avaliação contínua são essenciais para a validação e refinamento dos requisitos.

Desafio 3

Você é o líder de uma equipe de desenvolvimento de software e foi encarregado de implementar o RUP (Rational Unified Process) em um projeto de alta complexidade. Este método requer uma compreensão detalhada das diferentes fases do projeto e de como as atividades devem ser organizadas em cada fase. Sabendo que o número de iterações pode variar conforme a necessidade do projeto, como isso se aplica ao RUP e quais são as implicações dessa variação para a execução das atividades?

A

A disciplina Modelagem de Negócio apresenta maior atividade na fase de construção.

B

O número de iterações em cada uma de suas quatro fases é variável, conforme o projeto.

C

A disciplina Requisitos apresenta menor atividade na fase de concepção (Inception).

D

A disciplina de Teste não é executada na fase de Elaboração.

E

A disciplina Configuração e Gerenciamento de Mudanças não é executada na fase de Elaboração.



A alternativa B está correta.

A) A disciplina Modelagem de Negócio apresenta maior atividade na fase de construção: Incorreta. A Modelagem de Negócio é mais intensa nas fases iniciais, como a fase de Concepção, onde os requisitos e o escopo do projeto são definidos. Durante a Construção, o foco muda para a implementação e testes, e a Modelagem de Negócio já não é mais o foco principal.

B) O número de iterações em cada uma de suas quatro fases é variável, conforme o projeto: Correta. Esta é a resposta certa. No RUP, o número de iterações dentro de cada fase pode variar dependendo da complexidade e das necessidades específicas do projeto. Isso permite que o processo seja ajustado para melhor atender às demandas do projeto, proporcionando flexibilidade no desenvolvimento.

C) A disciplina Requisitos apresenta menor atividade na fase de concepção (Inception): Incorreta. Na fase de Concepção, a atividade de levantamento de requisitos é importante, pois é quando se definem os requisitos de alto nível e se estabelece o escopo do projeto. Portanto, essa disciplina tem grande atividade nesta fase.

D) A disciplina de Teste não é executada na fase de Elaboração: Incorreta. A disciplina de Teste é, sim, executada na fase de Elaboração, especialmente em protótipos e para validar os requisitos e o design. A ideia é detectar e corrigir problemas o quanto antes no processo.

E) A disciplina Configuração e Gerenciamento de Mudanças não é executada na fase de Elaboração: Incorreta. A Configuração e Gerenciamento de Mudanças é uma disciplina contínua que ocorre em todas as fases do RUP, incluindo a Elaboração. Ela é essencial para manter o controle das alterações e garantir a integridade do sistema.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Explicar o Processo Unificado para o desenvolvimento de software

“O Processo Unificado, também denominado de Rational Unified Process (RUP), é um processo de desenvolvimento de software criado pela Rational Software Corporation. O RUP possui três princípios: guiado por casos de uso, centrado em arquitetura e iterativo e incremental. Cada atividade do fluxo de trabalho se desenvolve de forma mais intensa em determinada fase.”

Desafio 4

Você está trabalhando como desenvolvedor em um projeto de software que utiliza metodologias ágeis, e foi encarregado de avaliar a aplicabilidade da XP (Extreme Programming) dentro da equipe. Durante o desenvolvimento, surgiram dúvidas sobre como as práticas de XP diferem de outras metodologias ágeis e quais são as principais exigências que ela impõe aos desenvolvedores. Considerando as práticas e princípios da XP, como você descreveria as exigências dessa metodologia em comparação com outras abordagens?

A

A XP (Extreme Programming), é considerada um processo de desenvolvimento de software ágil que exige entre outras coisas: padronização do código, interação contínua e teste.

B

É, na XP (Extreme Programming), sustentado por meio de pequenos e frequentes releases do sistema, e os clientes estão intimamente envolvidos na especificação e na priorização dos requisitos do sistema.

C

Enfoca, assim como o Acceptance Test-Driven Development, a qualidade do código desenvolvido quanto à recursividade, declaração das variáveis e clean code, de modo a torná-lo de fácil entendimento, modificação e testagem.

D

Pressupõe o uso do Behavior Driven Development, que considera a linguagem de programação a ser usada, da 4ª geração em diante, com foco, principalmente, no comportamento visual, interativo e cognitivo do sistema.

E

Enfoca a integração contínua como uma prática de desenvolvimento de software, incompatível com a XP (Extreme Programming) e o Scrum, que permite aos desenvolvedores agregarem alterações de código e realizarem testes.



A alternativa A está correta.

A) A XP (Extreme Programming), é considerada um processo de desenvolvimento de software ágil que exige entre outras coisas: padronização do código, interação contínua e teste: Correta. A XP é uma metodologia que enfatiza fortemente a qualidade do código e a eficiência do desenvolvimento. Ela exige padronização do código para garantir a clareza e a consistência, o que facilita a colaboração e a manutenção. A interação contínua e o teste são fundamentais para a XP, pois permitem o feedback constante e a correção de erros em tempo real, garantindo que o software evolua com qualidade.

B) É, na XP (Extreme Programming), sustentado por meio de pequenos e frequentes releases do sistema, e os clientes estão intimamente envolvidos na especificação e na priorização dos requisitos do sistema: Incorreta. Embora a XP realmente promova releases frequentes e a interação constante com o cliente, esta alternativa não aborda a padronização do código e a interação contínua como aspectos centrais, que são essenciais na metodologia XP.

C) Enfoca, assim como o Acceptance Test-Driven Development, a qualidade do código desenvolvido quanto à recursividade, declaração das variáveis e clean code, de modo a torná-lo de fácil entendimento, modificação e testagem: Incorreta. Esta alternativa menciona aspectos válidos, como a qualidade do código e o clean code, mas não se refere diretamente à padronização do código, interação contínua e testes como práticas específicas da XP. A recursividade, por exemplo, não é um foco exclusivo da XP.

D) Pressupõe o uso do Behavior Driven Development, que considera a linguagem de programação a ser usada, da 4ª geração em diante, com foco, principalmente, no comportamento visual, interativo e cognitivo do sistema: Incorreta. A XP não exige o uso do Behavior Driven Development (BDD) ou qualquer limitação à linguagem de programação de 4ª geração. Além disso, o foco da XP não está restrito ao comportamento visual ou cognitivo, mas sim na qualidade geral e simplicidade do código.

E) Enfoca a integração contínua como uma prática de desenvolvimento de software, incompatível com a XP (Extreme Programming) e o Scrum, que permite aos desenvolvedores agregarem alterações de código e realizarem testes: Incorreta. Esta alternativa é incorreta, pois a integração contínua é, na verdade, uma prática central da XP. Ela é compatível tanto com XP quanto com Scrum, e ambos os métodos a utilizam para garantir que as mudanças sejam continuamente integradas e testadas.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Explicar o processo de desenvolvimento ágil Extreme Programming (XP)

“A XP adota valores que servem como critérios que norteiam as partes interessadas de um projeto, incluindo: comunicação, simplicidade, feedback, coragem e respeito. A XP possui doze práticas que se enquadram nos valores e princípios descritos na tabela a seguir. O desenvolvimento orientado a testes, a padronização do código e a integração contínua são práticas essenciais para garantir a qualidade do software e a efetividade do processo de desenvolvimento.”

Qualidade de Software

Desafio 1

Você é um engenheiro de software responsável pela análise de qualidade em um grande projeto de desenvolvimento. Durante a avaliação das métricas utilizadas pela equipe, você deve identificar quais métricas de qualidade são mais apropriadas para garantir a manutenção e a eficiência do código desenvolvido. Seu objetivo é selecionar a métrica correta que permitirá uma avaliação eficaz do nível de complexidade e da compreensibilidade do software. Com base nesse cenário, escolha a opção correta que melhor se aplica à situação.

A

O número de funções ou métodos que constam em um programa pode ser avaliado pela métrica de software Fan-In/Fan-Out.

B

A métrica de complexidade ciclomática é uma medida que pode estar relacionada ao nível de compreensão do programa.

C

A medida do número de caracteres em um programa é uma métrica do tipo Fog Index.

D

A métrica de comprimento total faz referência ao número de linhas no código que se considera inversamente proporcional ao índice de erro que o código pode apresentar.

E

A métrica de profundidade de condições aninhadas é a que permite uma melhor compreensão do código.



A alternativa B está correta.

A) Incorreta. A métrica Fan-In/Fan-Out é útil para medir o acoplamento de um módulo, ou seja, quantas funções externas chamam uma função específica (Fan-In) e quantas outras funções são chamadas por ela (Fan-Out). Embora útil para avaliar o acoplamento, essa métrica não aborda diretamente a complexidade ciclomática, que está mais associada ao controle de fluxo dentro do código.

B) Correta. A métrica de complexidade ciclomática, introduzida por Thomas McCabe, mede a quantidade de caminhos lineares independentes em um programa. Essa métrica é importante para avaliar a complexidade do código, pois indica o número de decisões lógicas que afetam o fluxo de controle, permitindo assim uma melhor compreensão do nível de complexidade e da dificuldade de manutenção. Um código com alta complexidade ciclomática tende a ser mais difícil de entender e testar, o que pode impactar a qualidade geral do software.

C) Incorreta. O Fog Index é uma métrica de legibilidade que avalia a complexidade textual de um documento, baseada no número de palavras e frases complexas. Não se aplica diretamente à avaliação de código-fonte em termos de complexidade lógica ou estrutural.

D) Incorreta. A métrica de comprimento total refere-se ao número de linhas de código (LOC). Embora o número de linhas possa dar uma ideia da extensão do código, não está diretamente relacionado ao índice de erros. Códigos curtos podem ser muito complexos, enquanto códigos longos podem ser simples e fáceis de manter.

E) Incorreta. A profundidade de condições aninhadas se refere à quantidade de níveis de aninhamento de estruturas condicionais, como if-else, dentro do código. Embora possa impactar a legibilidade, não é a métrica mais eficaz para avaliar a compreensão geral do código como a complexidade ciclomática.

Para saber mais sobre esse conteúdo, acesse:

Módulo 4

Medições e métricas do software

“Comparando os valores medidos uns com os outros e com os padrões que se aplicam a uma organização, deve ser possível tirar conclusões sobre a qualidade do software ou avaliar a eficácia de processos de software, de ferramentas e de métodos, ou seja, o gerenciamento da qualidade pode contar com medições de atributos que afetam a qualidade do software.”

Desafio 2

Você está liderando uma equipe de desenvolvimento de software, e em momentos críticos de entrega, percebeu que a produtividade da equipe diminui e o estresse aumenta significativamente. Para mitigar esses efeitos, você decide automatizar certos processos dentro do ciclo de desenvolvimento. Sua tarefa é determinar a melhor abordagem para garantir a estabilidade e a continuidade das operações do software em produção, ao mesmo tempo em que mantém a qualidade do produto final. Qual das seguintes ações seria mais adequada para atingir esses objetivos?

A

Como parte da implantação contínua, ele automatiza o processo de forma que impeça que a nova versão do sistema entre em produção, caso o teste falhe.

B

Como parte da entrega contínua, ele automatiza uma série de rotinas para que o sistema seja automaticamente posto em produção.

C

Como parte da implantação contínua, ele elabora uma série de testes para garantir que ao implementar uma nova rotina ou funcionalidade, as outras partes do sistema continuem operando normalmente.

D

Como parte da integração contínua, ele automatiza o processo de agregar novas mudanças na forma de recursos e funcionalidades, em uma nova versão.

E

Como parte da entrega contínua, ele elabora smoke tests para garantir o funcionamento do sistema, antes de enviá-lo para produção.



A alternativa A está correta.

A) Correta. Automatizar o processo para impedir que uma nova versão entre em produção caso o teste falhe é essencial para garantir a qualidade do software. Esta prática é parte importante do ciclo de implantação contínua, onde a qualidade é assegurada em cada etapa, evitando que bugs ou falhas críticas cheguem ao ambiente de produção. A implantação contínua foca na automatização do processo de integração de novas funcionalidades, garantindo que o sistema se mantenha estável e funcional ao longo de várias iterações.

B) Incorreta. Automatizar a entrega contínua para colocar o sistema automaticamente em produção sem uma validação robusta pode aumentar os riscos de introduzir erros no ambiente de produção. A falta de uma verificação rigorosa antes da entrega pode comprometer a estabilidade do sistema.

C) Incorreta. Embora a criação de uma série de testes para garantir que novas funcionalidades não quebrem o sistema seja uma prática importante, esta ação sozinha não impede que uma versão com falhas entre em produção. A garantia de que o sistema só seja implantado se os testes forem bem-sucedidos é uma camada adicional de segurança necessária.

D) Incorreta. A integração contínua se concentra em agregar mudanças ao código de forma rápida e eficiente, mas não garante por si só que a nova versão esteja livre de problemas antes da implantação. Automatizar a integração sem o devido controle de qualidade pode levar à propagação de erros.

E) Incorreta. Os smoke tests são testes rápidos que verificam se o sistema é estável após pequenas mudanças, mas não são suficientes para garantir a ausência de falhas críticas. Confiar apenas em smoke tests antes da produção pode ser arriscado, pois eles não cobrem a profundidade necessária para assegurar a qualidade total.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Planejamento da qualidade e o controle da qualidade de software

“A equipe de SQA, ou equipe de garantia da qualidade, tem como atribuição auxiliar a equipe de software a obter um produto final de alta qualidade. No nível organizacional, o gerenciamento de qualidade envolve a aplicação de processos específicos de qualidade, verificando se os processos planejados foram seguidos, e a garantia de que as saídas de projeto estejam em conformidade com os padrões aplicáveis ao projeto.”

Desafio 3

Como engenheiro de software, você está envolvido na avaliação da qualidade de um projeto conforme os padrões internacionais. A norma ISO 9126 define vários atributos que devem ser considerados para garantir a qualidade do software. Diante da tarefa de selecionar os fatores de qualidade mais relevantes para o seu projeto, você deve identificar o atributo correto que se relaciona ao contexto de eficiência, usabilidade, manutenibilidade, funcionalidade ou confiabilidade do software. Qual das opções abaixo melhor representa a relação entre os atributos de qualidade e a descrição fornecida?

A

Eficiência, que representa a facilidade com que o software pode ser transferido entre ambientes computacionais.

B

Usabilidade, que representa o nível em que o software utiliza-se de recursos do sistema, de forma otimizada.

C

Manutenibilidade, que representa o período de tempo em que o software esteja disponível para uso.

D

Funcionalidade, que representa a facilidade com que o software pode ser transferido entre ambientes computacionais.

E

Confiabilidade, que representa o período de tempo em que o software esteja disponível para uso.



A alternativa E está correta.

A) Incorreta. A eficiência no contexto da qualidade de software se refere à relação entre o desempenho do software e os recursos utilizados, como o tempo de processamento e o uso da memória, não à facilidade de transferência entre ambientes computacionais.

B) Incorreta. Usabilidade se refere à facilidade de uso e à experiência do usuário ao interagir com o software, incluindo a facilidade de aprendizado e operação do sistema. Não está relacionada diretamente ao uso otimizado de recursos do sistema, que é mais uma questão de eficiência.

C) Incorreta. Manutenibilidade refere-se à facilidade com que o software pode ser modificado para corrigir defeitos, melhorar o desempenho ou adaptar-se a um ambiente modificado. Não está relacionada ao tempo de disponibilidade do software, que é mais uma questão de confiabilidade.

D) Incorreta. Funcionalidade diz respeito à adequação do software para suas funções pretendidas, como a exatidão e a conformidade com os requisitos especificados. A facilidade de transferência entre ambientes está relacionada à portabilidade, não à funcionalidade.

E) Correta. Confiabilidade é definida como a capacidade do software de realizar as funções esperadas de forma consistente e por um período específico de tempo. Isso inclui a disponibilidade do software para uso, que é importante para garantir que o software funcione corretamente quando necessário. A confiabilidade é um fator chave para determinar a estabilidade e a segurança do sistema em operação.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Planejamento da qualidade e o controle da qualidade de software

“A qualidade do produto pode ser garantida por meio de um sistemático plano de testes executado durante a etapa de codificação do processo de software, sendo esses testes denominados de testes de validação. Inicialmente, ocorre a validação da estrutura interna da unidade de software, bem como sua aderência aos requisitos estabelecidos.”

Desafio 4

Você está coordenando um projeto de software e precisa analisar o impacto das práticas de controle de qualidade nos custos totais do projeto. Ao considerar diferentes práticas, sua tarefa é escolher a abordagem mais eficaz para minimizar a ocorrência de erros durante o desenvolvimento, garantindo um produto final de alta qualidade. Dentre as alternativas a seguir, qual prática você considera mais adequada para alcançar esse objetivo?

A

Adoção de uma abordagem Waterfall.

B

Uso frequente de auditorias de qualidade.

C

Implementação tardia de testes de sistema.

D

Redução de revisões de código.

E

Limitação da interação com o cliente.



A alternativa B está correta.

A) Incorreta. A abordagem Waterfall é um modelo sequencial de desenvolvimento, onde cada fase deve ser concluída antes da próxima começar. Embora essa abordagem possa funcionar para projetos bem definidos, ela não é a mais eficaz para minimizar erros durante o desenvolvimento, pois não permite flexibilidade para revisões e ajustes contínuos.

B) Correta. O uso frequente de auditorias de qualidade é uma prática importante para identificar e corrigir falhas durante o processo de desenvolvimento. Auditorias regulares garantem que as práticas e os padrões de qualidade estejam sendo seguidos, o que ajuda a detectar e resolver problemas antes que eles se tornem críticos. Essa prática também assegura que o projeto esteja no caminho certo para entregar um produto final de alta qualidade, minimizando retrabalho e custos associados a correções de última hora.

C) Incorreta. A implementação tardia de testes de sistema pode levar à descoberta de falhas críticas muito tarde no processo de desenvolvimento, o que pode ser caro e demorado para corrigir. Testes devem ser implementados de forma contínua e antecipada para garantir a detecção precoce de problemas.

D) Incorreta. A redução de revisões de código pode comprometer a qualidade do software, pois essas revisões são essenciais para identificar erros, melhorar a qualidade do código e garantir que as melhores práticas de programação estejam sendo seguidas.

E) Incorreta. Limitar a interação com o cliente pode resultar em um produto final que não atende às expectativas e necessidades do usuário final. A comunicação contínua com o cliente é vital para garantir que o software desenvolvido corresponda aos requisitos e funcione como esperado.

Para saber mais sobre esse conteúdo, acesse:

Módulo 3

Planejamento da qualidade e o controle da qualidade de software

“O plano de qualidade de software, elaborado pela equipe de garantia da qualidade, fornece um roteiro para instituir a garantia da qualidade de software, devendo ser desenvolvido pela equipe SQA ou pela equipe de desenvolvimento, no caso da inexistência da equipe SQA. O plano serve como um gabarito para atividades de garantia da qualidade de software que serão instituídas para cada projeto de software.”

Gerenciamento de Configurações

Desafio 1

Imagine que você é um desenvolvedor de software responsável por um projeto em uma empresa de tecnologia. O desenvolvimento está em pleno andamento, mas a empresa está passando por várias mudanças nos requisitos do software, o que é comum em projetos dessa natureza. Como desenvolvedor, é importante que você entenda o processo de gerenciamento de configuração, que inclui a manutenção da integridade e consistência do software durante seu ciclo de vida. Esse processo abrange a identificação das versões corretas dos componentes, o controle das mudanças feitas no software, e a garantia de que todas as alterações sejam integradas de forma contínua e sem falhas. Com base nesse cenário, analise as opções abaixo e escolha a alternativa que melhor descreve os níveis de gerenciamento de configuração de software.

A

Controle de versão: identifica em qual versão está o software e quais as características de cada versão. Controle de mudanças: identifica quais foram as mudanças efetuadas na versão. Integração contínua: tem como características testar as mudanças, assim que são realizadas.

B

Controle de versão: identifica em qual versão está o software e quais as características de cada versão. Controle de mudanças: tem como característica testar as mudanças, assim que são realizadas. Integração contínua: identifica quais foram as mudanças efetuadas na versão.

C

Controle de versão: identifica quais foram as mudanças efetuadas na versão. Controle de mudanças: tem como característica testar as mudanças, assim que são realizadas. Integração contínua: identifica em qual versão está o software e quais as características de cada versão.

D

Controle de versão: identifica os usuários dos sistemas. Controle de mudanças: tem como característica testar as mudanças, assim que são realizadas. Integração contínua: identifica em qual versão está o software e quais as características de cada versão.

E

Controle de versão: identifica os usuários dos sistemas. Controle de mudanças: tem como característica testar as mudanças, assim que são realizadas. Integração contínua: identifica as aplicações do software.



A alternativa A está correta.

Alternativa A: Correta. Esta alternativa aborda corretamente os três níveis de gerenciamento de configuração. O controle de versão garante que as diferentes versões do software sejam identificadas e rastreadas adequadamente. O controle de mudanças assegura que todas as alterações sejam documentadas e testadas antes de serem integradas. A integração contínua verifica essas mudanças rapidamente, minimizando o risco de erros e garantindo a estabilidade do software. Este processo é vital para o sucesso de projetos de software, pois permite que as mudanças sejam implementadas de maneira organizada e sem comprometer a integridade do sistema.

Alternativa B: Incorreta. Aqui, a descrição de controle de mudanças e integração contínua está trocada. Controle de mudanças deve ser associado à identificação e registro das modificações, enquanto a integração contínua é o processo de testar essas mudanças para garantir sua correta implementação.

Alternativa C: Incorreta. Esta opção confunde os conceitos ao inverter as descrições de controle de versão e integração contínua. O controle de versão não se limita a identificar as mudanças realizadas; ele é responsável por rastrear as diferentes versões do software, enquanto a integração contínua cuida da aplicação dessas mudanças de maneira segura.

Alternativa D: Incorreta. Esta alternativa descreve incorretamente o controle de versão como a identificação dos usuários, o que não faz parte do escopo do controle de versão. O controle de versão está relacionado à gestão das versões do software, enquanto a identificação de usuários está mais ligada à gestão de acesso.

Alternativa E: Incorreta. Além do erro similar à alternativa D, esta opção descreve incorretamente a integração contínua, associando-a à identificação de aplicações do software, o que não corresponde ao objetivo deste processo.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Processo de Gerenciamento de Configurações de Software

“O processo de gerenciamento de configurações de software (GCS), proposto por Pressman (2016), inclui tarefas que têm quatro objetivos primários: identificar todos os itens que coletivamente definem a configuração do software, gerenciar alterações de um ou mais desses itens, facilitar a construção de diferentes versões de uma aplicação, e assegurar que a qualidade do software seja mantida à medida que a configuração evolui com o tempo.”

Desafio 2

Imagine que você é um engenheiro de software responsável pelo desenvolvimento de um sistema que será utilizado por diversas equipes em uma grande empresa. A garantia de que o sistema evolua de forma controlada, sem que as mudanças feitas por diferentes desenvolvedores interfiram negativamente umas nas outras, é importante para o sucesso do projeto. O gerenciamento de versões e o controle dessas alterações são partes essenciais desse processo, permitindo que a equipe trabalhe de maneira colaborativa e segura, mantendo a integridade e a funcionalidade do software. Considerando essa situação, avalie as afirmações a seguir relacionadas ao gerenciamento de versões em um sistema de software.

I- O gerenciamento de versões é o processo de acompanhamento de diferentes versões de componentes de software ou itens de configuração e dos sistemas em que esses componentes são usados. Refere-se também à garantia de que as mudanças feitas por diferentes desenvolvedores para essas versões não interfiram umas nas outras.

II- Em um sistema de gerenciamento de versões com uma variedade de recursos, desenvolvedores diferentes não podem trabalhar, ao mesmo tempo, no mesmo componente, pois se o fizessem, as mudanças feitas por diferentes desenvolvedores poderiam interferir umas nas outras.

III- Os sistemas de gerenciamento de versões fornecem, em geral, recursos de gerenciamento de armazenamento. Sua função é reduzir o espaço de armazenamento requerido pelas várias versões de componentes, que diferem apenas ligeiramente umas das outras. Em vez de manter uma cópia completa de cada versão, o sistema armazena uma lista de diferenças (deltas) entre uma versão e outra.

A

Apenas II.

B

Apenas I e II.

C

Apenas I e III.

D

Apenas II e III.

E

I, II e III.



A alternativa C está correta.

O gerenciamento de versões é um aspecto importante na engenharia de software, permitindo que diferentes versões de componentes sejam gerenciadas de forma eficaz, garantindo a integridade do sistema. A primeira afirmação está correta ao definir o gerenciamento de versões como o processo que acompanha diferentes versões e assegura que as mudanças feitas por diversos desenvolvedores não interfiram negativamente entre si. Esta prática é essencial para manter a qualidade e a estabilidade do software ao longo de seu desenvolvimento.

A terceira afirmação também é correta ao explicar que os sistemas de gerenciamento de versões economizam espaço de armazenamento utilizando listas de diferenças (deltas) entre versões, em vez de manter cópias completas de cada versão. Esse método é eficiente, pois permite gerenciar múltiplas versões de software sem consumir grande quantidade de armazenamento.

Por outro lado, a segunda afirmação está incorreta. Em sistemas modernos de gerenciamento de versões, como o Git, diferentes desenvolvedores podem trabalhar simultaneamente no mesmo componente, com o

sistema lidando com as fusões de alterações de maneira a evitar conflitos, ao contrário do que a afirmação sugere.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Gerenciamento de mudanças, versões e releases.

"O gerenciamento de versões tem como objetivo controlar as diferentes versões de componentes de software e dos sistemas, envolvendo também a garantia de que as alterações feitas por diferentes desenvolvedores não interfiram umas com as outras... Os sistemas de gerenciamento de versões fornecem, em geral, recursos de gerenciamento de armazenamento. Sua função é reduzir o espaço de armazenamento requerido pelas várias versões de componentes, que diferem apenas ligeiramente umas das outras. Em vez de manter uma cópia completa de cada versão, o sistema armazena uma lista de diferenças (de...)".

Desafio 3

Você está trabalhando como engenheiro de software em uma equipe que utiliza diferentes sistemas de controle de versão para gerenciar o código-fonte de seus projetos. Recentemente, houve uma discussão sobre as diferenças entre os sistemas centralizados, como o SVN, e os sistemas distribuídos, como o Git. É importante para você e sua equipe entenderem as vantagens e limitações de cada sistema, especialmente no que diz respeito à recuperação de dados e ao gerenciamento de versões. Com base nesse contexto, avalie as afirmações abaixo e selecione a alternativa correta.

A

O svn utiliza um modelo centralizado para o versionamento de sistemas, enquanto o git apresenta um modelo distribuído, contendo cópias do repositório original em cada cliente.

B

Uma das vantagens em se utilizar um sistema de controle de versão centralizado está no fato de ser possível recuperar o conteúdo original a partir dos repositórios presentes nos clientes.

C

Uma das características dos sistemas de controle de versão é controlar de forma anônima as alterações que são realizadas no sistema versionado, obtendo somente um controle das alterações feitas no decorrer do tempo.

D

Em um sistema de controle de versão distribuído, há um único servidor contendo todos os arquivos versionados. Os clientes obtêm somente parte do repositório armazenado no servidor, não possuindo a sua visão completa.

E

Dado seu comportamento centralizado, o git possui a desvantagem de ter um único ponto de falha, onde uma falha no servidor central ocasionará a perda completa do repositório, não sendo possível recuperar a partir do conteúdo existente nos clientes.



A alternativa A está correta.

Alternativa A: Correta. Esta alternativa descreve corretamente as diferenças fundamentais entre SVN e Git. O SVN utiliza um modelo centralizado onde todo o controle de versão é gerenciado por um único servidor, enquanto o Git é um sistema distribuído em que cada desenvolvedor tem uma cópia completa do repositório em seu sistema local. Isso permite que o Git ofereça uma maior resiliência e flexibilidade, pois o trabalho pode continuar mesmo se o servidor central estiver offline.

Alternativa B: Incorreta. Essa afirmação é equivocada porque descreve uma característica que é típica dos sistemas distribuídos, como o Git, e não dos sistemas centralizados, como o SVN. Em um sistema distribuído, as cópias locais dos desenvolvedores contêm todo o histórico do repositório, permitindo a recuperação de dados mesmo se o servidor principal falhar.

Alternativa C: Incorreta. Sistemas de controle de versão, como SVN e Git, não controlam de forma anônima as alterações. Pelo contrário, eles rastreiam detalhadamente quem fez cada mudança, quando e por quê. Essa informação é importante para o gerenciamento eficaz do desenvolvimento de software e para a resolução de problemas.

Alternativa D: Incorreta. Esta alternativa confunde os conceitos entre sistemas distribuídos e centralizados. Em sistemas distribuídos como o Git, não há dependência de um único servidor; cada desenvolvedor tem uma visão completa do repositório. A descrição fornecida se aplica mais a um sistema centralizado como o SVN.

Alternativa E: Incorreta. Essa afirmação está errada ao descrever o Git, que é um sistema distribuído, como se fosse centralizado. Em um sistema distribuído como o Git, não há um único ponto de falha, pois cada desenvolvedor possui uma cópia completa do repositório, permitindo que o trabalho continue independentemente do estado do servidor central.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Gerenciamento de Versões

“O gerenciamento de versões tem como objetivo controlar as diferentes versões de componentes de software e dos sistemas, envolvendo também a garantia de que as alterações feitas por diferentes desenvolvedores não interfiram umas com as outras. De acordo com Sommerville (2019), o gerenciamento de versões é o processo de gerenciamento de codelines e baselines.”

Desafio 4

Como desenvolvedor de software, você faz parte de uma equipe que utiliza o Git para gerenciar o código-fonte dos projetos. Recentemente, você precisou incorporar atualizações de um repositório remoto para a sua cópia local e se deparou com o comando "pull". Esse comando é importante para sincronizar seu trabalho com as alterações feitas por outros membros da equipe. Dada a importância de entender como o comando "pull"

funciona no contexto do Git, avalie as opções abaixo e escolha a alternativa que descreve corretamente essa funcionalidade.

A

É uma solicitação para baixar localmente os arquivos do repositório remoto do usuário.

B

É o envio de arquivos para o repositório remoto do usuário.

C

É o envio de arquivos para o repositório local do usuário.

D

É uma solicitação que um usuário faz para contribuir com suas alterações de volta ao repositório original de um projeto.

E

Atualiza o repositório local com todas as alterações feitas em outro repositório.



A alternativa E está correta.

Alternativa A: Incorreta. Embora a descrição pareça se aproximar do que o comando "pull" realiza, ela é imprecisa. O comando "pull" não apenas baixa os arquivos; ele também os mescla com a cópia local do repositório, garantindo que todas as mudanças sejam aplicadas de forma integrada.

Alternativa B: Incorreta. Essa alternativa descreve a funcionalidade do comando "push", que é o oposto do "pull". O "push" envia as alterações locais para o repositório remoto, enquanto o "pull" traz as alterações do remoto para o local.

Alternativa C: Incorreta. Essa opção descreve parcialmente o que o "pull" faz, mas falta clareza. O "pull" faz mais do que simplesmente enviar arquivos; ele sincroniza completamente o repositório local com as mudanças no repositório remoto.

Alternativa D: Incorreta. A descrição corresponde ao conceito de um "pull request", que é uma solicitação para integrar mudanças em um projeto, mas isso é diferente do comando "pull". Um "pull request" é uma prática de colaboração, enquanto o "pull" é um comando de sincronização.

Alternativa E: Correta. Esta é a descrição mais precisa do comando "pull" no Git. O "pull" atualiza o repositório local com todas as alterações feitas em um repositório remoto, garantindo que o desenvolvedor trabalhe sempre com a versão mais atualizada do projeto. Esse comando é essencial para manter a coesão e a integridade do código quando se trabalha em equipe.

Para saber mais sobre esse conteúdo, acesse:

Módulo 2

Gerenciamento de Versões

“Em vez de simplesmente fazer o check-out dos arquivos de que precisa, um desenvolvedor cria um clone do repositório do projeto, que é baixado e instalado em seu computador. Quando terminam de fazer as alterações, realizam um commit nas mesmas, atualizando seu repositório. A disponibilização dessas alterações para o repositório do projeto ocorre por meio de uma operação push, cabendo ao gerente de configuração ‘puxar’ (pull) esses arquivos para o repositório do projeto, ou seja, um repositório pode se comunicar com qualquer outro através das operações básicas pull e push.”

Considerações finais

Continue explorando, praticando e desafiando-se. Cada exercício é uma oportunidade de crescimento e cada erro, uma lição valiosa. Que sua jornada de aprendizado seja repleta de descobertas e realizações. Bons estudos e sucesso na sua carreira!

Compartilhe conosco como foi sua experiência com este conteúdo. Por favor, responda a este [formulário de avaliação](#) e nos ajude a aprimorar ainda mais a sua experiência de aprendizado!